

8강

반복문





1. 반복문이란 ?

프로그램 진행을 특정 조건에 따라 반복적으로 진행하는 것이다.

for, while문 : 조건이 참일 때까지 반복 수행

[예1]

구구단을 구하기 위해서 1에서부터 1씩 더하면서 9까지 곱셈 연산을 진행한다.

[예2]

조도 센서를 센싱한 데이터가 10미만이면 건물의 LED를 1초 간격으로 계속 점등한다.



2. While문 ?

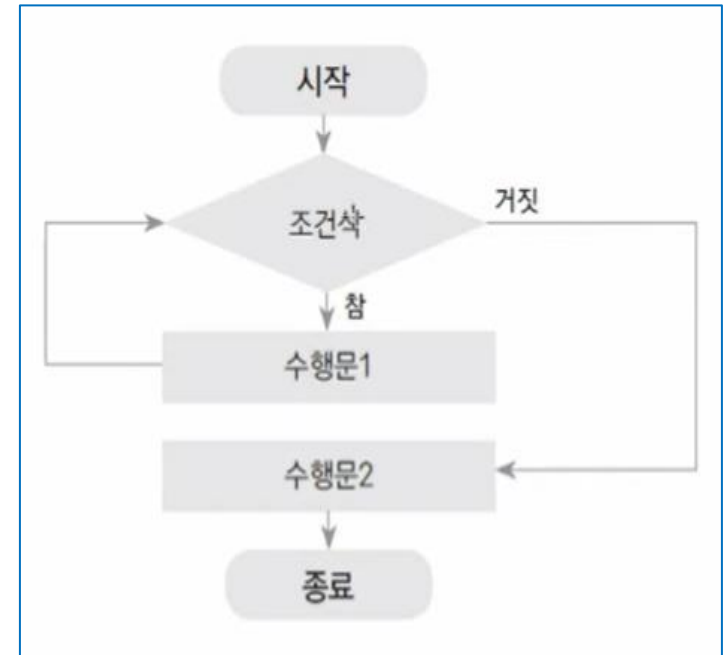
- while문은 조건이 참(true)이면 반복해서 작업을 수행 한다.
- 중간에 while문을 빠져 나와야 하는 경우는 우선 break를 만나면 while문을 빠져 나오게 할 수 있다.

```
while(조건식) {
```

반복 실행 할 수행 문;

```
}
```

조건이 참인 동안
반복 수행





➤ while문 간단한 예제

```
public class WhileExample01 {  
  
    public static void main(String[] args) {  
  
        int num = 1;  
        int sum = 0; // 반드시 초기화  
  
        while(num<=10) {  
  
            sum += num;  
            num++; // 구문이 없으면 무한 루프 발생  
  
        }  
  
        System.out.println("1부터 10까지의 합 : "+sum);  
    }  
}
```

반복 실행 할 구문

무한 루프 실행 됨

```
while(true) {  
  
    sum += num;  
    num++; // 구문이 없으면 무한루프 발생  
    break;  
}
```

루프를 빠져 나옴



➤ while문 간단한 예제

while(rNum < 10)

rNum이 10보다 작을 때 까지
프로그램 반복 진행

```
// while문
System.out.print("INPUT NUMBER : ");
int num = scanner.nextInt();
int i = 1;
while (i < 10) {
    System.out.printf("%d * %d = %d\n", num, i, (num * i));
    i++;
}
```



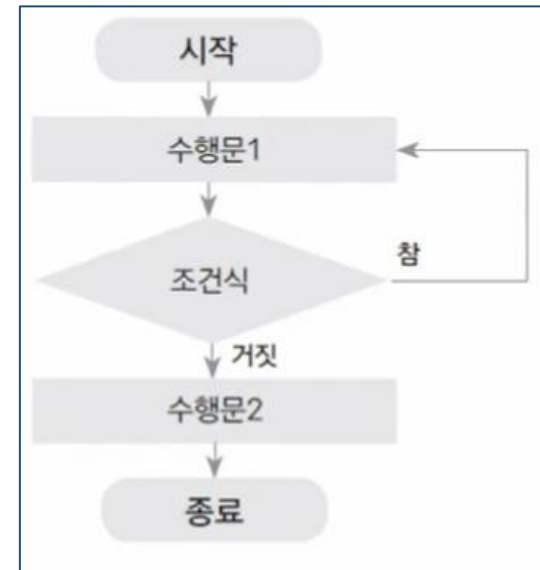
```
INPUT NUMBER : 9
9 * 1 = 9
9 * 2 = 18
9 * 3 = 27
9 * 4 = 36
9 * 5 = 45
9 * 6 = 54
9 * 7 = 63
9 * 8 = 72
9 * 9 = 81
```



3. Do ~ While 설명

- do~while문은 while문과 비교해서 조건과 상관 없이 무조건 한 번 작업을 수행한다.
- 그 다음 조건이 참(true)이면 반복해서 작업을 수행 한다.
- while문의 경우 조건이 만족하지 않는다면 한번도 반복하지 않을 수 있다.하지만, do while문의 경우는 무조건 한번은 실행되는 반복문 이다.

```
do{  
  
    실행문;  
  
} while(조건문);
```





➤ do~ While 간단한 예제

```
do {  
    sum += num;  
    num++; // 구문이 없으면 무한루프 발생  
}while(num <= 10);  
  
System.out.println("1부터 10까지의 합: "+sum);  
}
```

무조건 한 번은 실행 됨

```
// do ~ while문  
do {  
    System.out.println("무조건 1번은 실행합니다.");  
} while (false);
```

무조건 1번은 실행합니다.



4. For문 설명

- 반복문은 어떤 작업이 반복적으로 수행되도록 할 때 사용된다.
- For문은 주로 반복횟수를 알고 있을 때 사용한다.
- 초기화, 조건식, 증감식, 블록 모두 4부분으로 구성
- 조건식이 참인 동안 블록 내의 문장들을 반복하다 거짓이면 반복문을 벗어난다.



```
for(초기화; 조건식; 증감식) {  
    조건식이 참일 때 수행되는 부분  
}
```




4. For문 수행 과정

Num이 1에서 부터 5일 때 까지 하나씩 증가하면서 출력하는 for문

```
int num;
for(num = 1; num <= 5; num++)
{
    System.out.println(num);
}
```

- ① 1초기화는 한번만 수행 하고
- ② 조건식이 만족하면
- ③ 수행문 수행
- ④ 증감식 수행
- ⑤ 조건식이 만족하는지 체크
- ⑥ 조건식이 만족하면 수행문 수행
- ⑦ 조건식이 만족하지 않으면 for문 빠져 나옴

num 값	1(초기화)	2	3	4	5	6
조건식 (num <= 5)	참	참	참	참	참	거짓
출력 값	1	2	3	4	5	for문 종료
증감식	수행	수행	수행	수행	수행	x



➤ For 간단한 예제01

두 가지 방식 모두 사용가능

```
int num ;  
int sum ;  
for(num=1,sum=0;num<=10;num++) {  
  
    sum += num;  
    System.out.printf("sum: %d,num: %d%n",sum,num);  
  
}  
  
System.out.println(sum);  
System.out.println(num);  
}
```

```
int sum = 0;  
for(int num=1;num<=10;num++) {  
  
    sum += num;  
  
}  
System.out.println(sum);  
}
```



➤ For 간단한 예제02

```
for(int i = 0; i < 10; i++) {...}
```

```
for (int i = 1; i < 10; i++)
```

i가 1부터 10보다 작을 때 까지 i에
1씩 더해가며 프로그램 반복 진행

```
for (int i = 1; i < 10; i = i + 2)
```

i가 1부터 10보다 작을 때 까지 i에
2씩 더해가며 프로그램 반복 진행

// for문

```
System.out.print("INPUT NUMBER : ");  
Scanner scanner = new Scanner(System.in);  
int inputNum = scanner.nextInt();
```

```
for (int i = 1; i < 10; i++) {  
    System.out.printf("%d * %d = %d\n", inputNum, i, (inputNum * i));  
}
```



```
<terminated> MainClass (/)  
INPUT NUMBER : 4  
4 * 1 = 4  
4 * 2 = 8  
4 * 3 = 12  
4 * 4 = 16  
4 * 5 = 20  
4 * 6 = 24  
4 * 7 = 28  
4 * 8 = 32  
4 * 9 = 36
```



➤ 중첩된 반복문

반복문 내부에 또 반복문이 사용 됨
구구단의 예

2단부터 9단까지 반복하는 외부 반복문

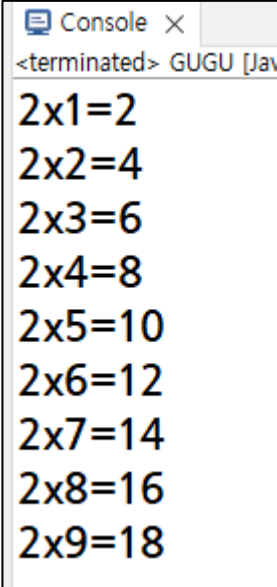
```
for(dan = 2; dan <= 9; dan++) {  
    for(times = 1; times <= 9; times++) {  
        System.out.println(dan + "X" + times + "=" + dan * times);  
    }  
    System.out.println( ); //한 줄 띄워서 출력  
}  
}
```

각 단에서 1~9를 곱하는 내부 반복문



➤ 중첩된 반복문 – while 로 작성 시

```
public static void main(String[] args) {  
  
    int dan = 2;  
    int times = 1;  
  
    while( dan<=9 ) {  
  
        while( times<=9) {  
            System.out.println(dan + "x" + times + "=" + dan*times);  
            times ++;  
        }  
  
        dan ++ ;  
        System.out.println();  
    }  
}
```



Console ×
<terminated> GUGU [Jav
2x1=2
2x2=4
2x3=6
2x4=8
2x5=10
2x6=12
2x7=14
2x8=16
2x9=18



➤ 중첩된 반복문 – while 로 작성 시

```
public static void main(String[] args) {  
  
    int dan = 2;  
    int times = 1;  
  
    while( dan<=9 ) {  
        times = 1; // 반드시 초기화 한다.  
        while( times<=9 ) {  
            System.out.println(dan + "x" + times + "=" + dan*times);  
            times ++;  
        }  
        dan ++ ;  
        System.out.println();  
    }  
}
```



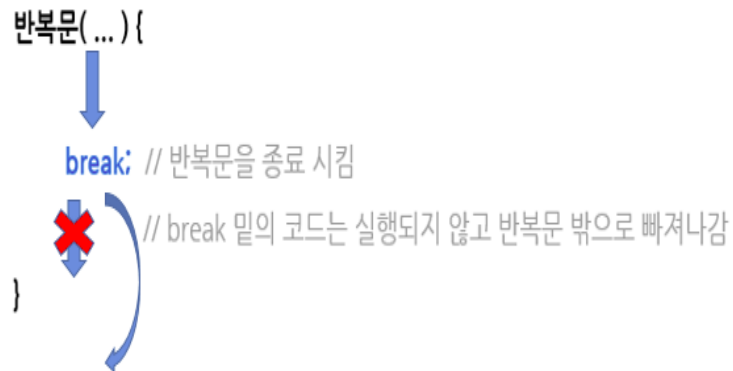
```
Console x  
<terminated> GUGU [Java A  
2x1=2  
2x2=4  
2x3=6  
2x4=8  
2x5=10  
2x6=12  
2x7=14  
2x8=16  
2x9=18  
  
3x1=3  
3x2=6  
3x3=9  
3x4=12  
3x5=15  
3x6=18  
3x7=21  
3x8=24  
3x9=27  
  
4x1=4  
4x2=8  
4x3=12  
4x4=16  
4x5=20  
4x6=24  
4x7=28  
4x8=32
```



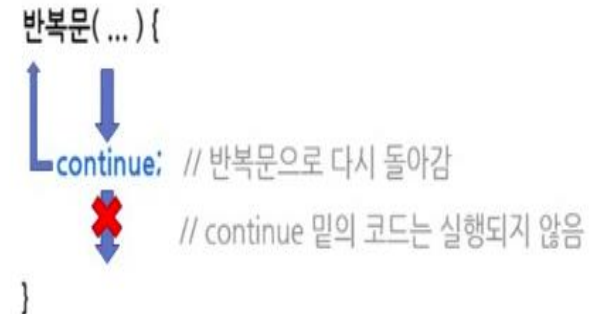
5. 보조 제어문 : break, continue

- **break** : 반복문 1개를 즉시 종료
- **continue** : 반복문의 조건식으로 다시 이동한다.

break



continue





➤ Break 문 간단한 예제

0부터 시작하여 1씩 늘리며 숫자의 합이 100이 초과하는 경우 그 수와 합을 구하는 예제

```
public class BreakExample2 {  
    public static void main(String[] args) {  
        int sum = 0;  
        int num = 0;  
  
        for(num = 0; ; num++) {  
            sum += num;  
            if(sum >= 100)           // sum이 100보다 크거나 같을 때(종료 조건)  
                break;              // 반복문 중단  
        }  
        System.out.println("num : " + num);  
        System.out.println("sum : " + sum);  
    }  
}
```

조건식을 생략하는 대신 break문을 사용합니다.

Problems Javadoc Declaration Console

<terminated> BreakExample2 [Java Application] C:\Program Files\Java\jre-10.0.1\bin\javaw.exe

num : 14
sum : 105



➤ Break 문 간단한 예제

i 가 4일 때 for 문을 빠져나온다.

```
for (int i = 0; i < 10; i++)  
{  
    if (i == 4) {  
        break;  
    }  
  
    System.out.println(i);  
}
```



The screenshot shows a Java IDE console window titled "Console". The output of the program is displayed as follows:

```
<terminated> BreakEx01 [Java]  
0  
1  
2  
3
```



➤ Continue 문 간단한 예제

- 반복문과 함께 쓰이며, 반복문 내부 continue 문을 만나면 이후 반복되는 부분을 수행하지 않고 조건식이나 증감식을 수행함

1부터 100까지 홀수만 더하는 예

```
public class ContinueExample {  
    public static void main(String[ ] args) {  
        int total = 0;  
        int num;  
  
        for(num = 1; num <= 100; num++) {  
            if(num % 2 == 0) {  
                continue;  
            }  
            total += num;  
        }  
        System.out.println("1부터 100까지의 홀수의 합은: " + total + "입니다.");  
    }  
}
```

// 100까지 반복
// num 값이 짝수인 경우
// 이후 수행을 생략하고 num++ 수행
// num 값이 홀수인 경우에만 수행



➤ Continue 문 간단한 예제

i 가 4일 때 아래 출력문을 수행하지 않고, for 문의 증감식으로 이동한다.

```
for (int i = 0; i < 10; i++) {  
    if (i == 4) {  
        continue;  
    }  
    System.out.println(i);  
}
```



```
<terminated> BreakEx01 [Java Application]  
0  
1  
2  
3  
5  
6  
7  
8  
9
```

연습문제





➤ 여기서 잠깐 !

- example패키지안에 LoopEx01.java로 class 파일을 작성하고 아래 <출력 결과>와 같이 출력되도록 코드를 작성하시오.

<문제> While 문 이용할 것

```
// 문제1) 1~10까지 반복해 5~9 출력  
// 정답1) 5, 6, 7, 8, 9
```

```
// 문제2) 10~1까지 반복해 6~3 거꾸로 출력  
// 정답2) 6, 5, 4, 3
```

```
// 문제3) 1~10까지 반복해 짝수만 출력  
// 정답3) 2, 4, 6, 8, 10
```



➤ 여기서 잠깐 !

- example패키지안에 LoopEx02.java로 class 파일을 작성하고 아래 <출력 결과>와 같이 출력되도록 코드를 작성하시오.

<문제> While 문 이용할 것

```
// 문제1) 1~5까지의 합 출력  
// 정답 1) 15
```

```
// 문제2) 1~10까지 반복해 3미만 7이상만 출력  
// 정답2) 1, 2, 7, 8, 9, 10
```

```
// 문제3) 문제2의 조건에 맞는 수들의 합 출력  
// 정답3) 37
```

```
// 문제4) 문제2의 조건에 맞는 수들의 개수 출력  
// 정답4) 6
```



➤ 여기서 잠깐 !

- example패키지안에 LoopEx03.java로 class 파일을 작성하고 아래 <출력 결과>와 같이 출력되도록 코드를 작성하시오.

<문제>구구단 게임, While 문 이용할 것

1. 구구단 게임을 5회 반복한다.
2. 정답을 맞추면 20점이다.
3. 게임 종료 후, 성적을 출력한다.

<출력 결과 물>

```
7 X 7 = 49
7 X 9 = 60
9 X 1 = 9
3 X 2 = 8
9 X 5 = 45
성적 = 60점
```



➤ 여기서 잠깐 !

- example패키지안에 LoopEx04.java로 class 파일을 작성하고 아래 <출력 결과>와 같이 출력되도록 코드를 작성하시오.

<문제> while 반복문 종료(-100)

1. 무한 반복을 하면서 숫자를 입력 받는다.
2. 입력한 숫자가 -100이면, 프로그램은 종료된다.



➤ 여기서 잠깐 !

- example패키지안에 LoopEx05.java로 class 파일을 작성하고 아래 <출력 결과>와 같이 출력되도록 코드를 작성하시오.

<문제> Up & Down 게임[2단계]

1. 정답을 맞추면 게임은 종료된다.
2. 100점부터 시작해 오답을 입력할 때마다 5점씩 차감된다.
3. 게임 종료 후, 점수를 출력한다.
4. 1 ~ 100 까지의 정수가 Random하게 출력되도록 할 것



➤ 여기서 잠깐 !

- example패키지안에 LoopEx06.java로 class 파일을 작성하고 아래 <출력 결과>와 같이 출력되도록 코드를 작성하시오.

<문제>영수증 출력[2단계]

1. 5번 주문을 받는다.
2. 주문이 끝난 후, 돈을 입력 받는다.
3. 각 메뉴 별 주문 수량과 총 금액을 출력한다.

4. 각 메뉴의 가격은 아래와 같다.

불고기버거 : 8700

새우버거 : 6200

콜라 : 1500

<출력 결과 물>

```
=== 롯데리아 ===  
1. 불고기 버거 : 8700원  
2. 새우 버거 : 6200원  
3. 콜라 : 1500원  
메뉴 선택 : 1  
메뉴 선택 : 2  
메뉴 선택 : 3  
메뉴 선택 : 3  
메뉴 선택 : 1  
총 금액 = 26600원  
현금 입력 : 30000
```

```
=== 롯데리아 영수증 ===  
1. 불고기 버거 : 2개  
2. 새우 버거 : 1개  
3. 콜라 : 2개  
4. 총 금액 : 26600원  
5. 잔돈 : 3400원
```



➤ 여기서 잠깐 !

- example패키지안에 LoopEx07.java로 class 파일을 작성하고 아래 <출력 결과>와 같이 출력되도록 코드를 작성하시오.

<문제>ATM[2단계]

1. 로그인

- . 로그인 후 재 로그인 불가
- . 로그아웃 상태에서만 이용 가능

2. 로그아웃

- . 로그인 후 이용가능

3. 서로 다른 두명의 계좌번호와 비밀번호는 아래와 같다.

```
int dbAcc1 = 1111;  
int dbPw1 = 1234;
```

```
int dbAcc2 = 2222;  
int dbPw2 = 2345;
```

<출력 결과 물>

```
1.로그인  
2.로그아웃  
0.종료  
메뉴 선택 : 1  
계좌번호 입력 : 1111  
비밀번호 입력 : 1234  
1111님, 환영합니다.  
1.로그인  
2.로그아웃  
0.종료  
메뉴 선택 : 2  
로그아웃 되었습니다.
```

```
1.로그인  
2.로그아웃  
0.종료  
메뉴 선택 : 0  
프로그램 종료
```



➤ 여기서 잠깐 !

- example패키지안에 LoopEx08.java로 class 파일을 작성하고 아래 <출력 결과>와 같이 출력되도록 코드를 작성하시오.

<문제>ATM[2단계]

1. 입금
 - . 입금할 금액을 입력받아, myMoney에 입금
2. 출금
 - . 출금할 금액을 입력받아, myMoney에서 출금
 - . 단, 출금할 금액이 myMoney 잔액을 초과할 경우 출금불가
3. 이체
 - . yourAcc 계좌번호를 입력받아, 이체
 - . 이체할 금액이 myMoney 잔액을 초과할 경우 이체 불가
 - . 이체 후 yourMoney 잔액 증가
4. 조회
 - . myMoney와 yourMoney 잔액 모두 출력

```
myAcc = 1111;  
myMoney = 50000;
```

```
yourAcc = 2222;  
yourMoney = 70000;
```

<출력 결과 물>

```
1.입금  
2.출금  
3.이체  
4.조회  
0.종료  
메뉴 선택 : 4  
myMoney = 50000원  
yourMoney = 70000원  
1.입금  
2.출금  
3.이체  
4.조회  
0.종료  
메뉴 선택 :
```



<문제8>의 결과 화면

```

1.입금
2.출금
3.이체
4.조회
0.종료
메뉴 선택 : 4
myMoney = 50000원
yourMoney = 70000원
1.입금
2.출금
3.이체
4.조회
0.종료
메뉴 선택 :
  
```

```

1.입금
2.출금
3.이체
4.조회
0.종료
메뉴 선택 : 2
출금할 금액 입력 : 10000
출금을 완료하였습니다.
1.입금
2.출금
3.이체
4.조회
0.종료
메뉴 선택 : 4
myMoney = 40000원
yourMoney = 70000원
  
```

```

1.입금
2.출금
3.이체
4.조회
0.종료
메뉴 선택 : 3
이체할 계좌번호 입력 : 2222
이체할 금액 입력 : 10000
이체를 완료하였습니다.
1.입금
2.출금
3.이체
4.조회
0.종료
메뉴 선택 :
  
```

```

1.입금
2.출금
3.이체
4.조회
0.종료
메뉴 선택 : 4
myMoney = 30000원
yourMoney = 80000원
1.입금
2.출금
3.이체
4.조회
0.종료
메뉴 선택 :
  
```



➤ 여기서 잠깐 !

- example패키지안에 LoopEx09.java로 class 파일을 작성하고 아래 <출력 결과>와 같이 출력되도록 코드를 작성하시오.

<문제> ATM[2단계]

- 로그인
 - . 로그인 후 재 로그인 불가
 - . 로그아웃 상태에서만 이용 가능
- 로그아웃
 - . 로그인 후 이용가능
- 입금
 - . 로그인 후 이용가능
- 출금
 - . 로그인 후 이용가능
- 이체
 - . 로그인 후 이용가능
- 조회
 - . 로그인 후 이용가능
- 종료
- 아래 변수를 이용 할 것

```
int dbAcc1 = 1111;
int dbPw1 = 1234;
int dbMoney1 = 50000;
```

```
int dbAcc2 = 2222;
int dbPw2 = 2345;
int dbMoney2 = 70000;
```

<출력 결과 물>

```
dbMoney1 = 50000원
dbMoney2 = 70000원
*상태 : 로그아웃
1.로그인
2.로그아웃
3.입금
4.출금
5.이체
6.조회
0.종료
메뉴 선택 :
```

```
*상태 : 1111로그인
1.로그인
2.로그아웃
3.입금
4.출금
5.이체
6.조회
0.종료
메뉴 선택 : 1
이미 로그인 중...
```

```
메뉴 선택 : 1
계좌번호 입력 : 1111
비밀번호 입력 : 1234
1111님, 환영합니다.
dbMoney1 = 50000원
dbMoney2 = 70000원
*상태 : 1111로그인
1.로그인
2.로그아웃
3.입금
4.출금
5.이체
6.조회
0.종료
메뉴 선택 :
```



<출력 결과 물>

메뉴 선택 : 3
 입금할 금액 입력 : 10000
 입금을 완료하였습니다.

dbMoney1 = 60000원
 dbMoney2 = 70000원
 *상태 : 1111로그인

- 1.로그인
- 2.로그아웃
- 3.입금
- 4.출금
- 5.이체
- 6.조회
- 0.종료

메뉴 선택 :

메뉴 선택 : 5
 이체할 계좌번호 입력 : 2222
 이체할 금액 입력 : 15000
 이체를 완료하였습니다.

dbMoney1 = 45000원
 dbMoney2 = 85000원
 *상태 : 1111로그인

- 1.로그인
- 2.로그아웃
- 3.입금
- 4.출금
- 5.이체
- 6.조회
- 0.종료

메뉴 선택 :

메뉴 선택 : 2
 로그아웃 되었습니다.

dbMoney1 = 45000원
 dbMoney2 = 85000원
 *상태 : 로그아웃

- 1.로그인
- 2.로그아웃
- 3.입금
- 4.출금
- 5.이체
- 6.조회
- 0.종료

메뉴 선택 : 0
 프로그램 종료